

CineMatch

A Content-Based Movie Recommendation System

Project Report — VITyarthi "Build Your Own Project"

Course Domain: AI / Machine Learning

Submitted by: FROST

Repository: <https://github.com/{github-username}/{repo-name}>

Technique used: TF-IDF Vectorization + Cosine Similarity

1. Introduction

CineMatch is a command-line AI/ML application that recommends movies similar to one a user already likes. It demonstrates a complete, correctly-engineered content-based filtering pipeline: raw text data (genres and plot overviews) is converted into numeric TF-IDF vectors, and recommendations are produced by finding the nearest neighbours of a chosen movie in that vector space using cosine similarity. The project favours a small, fully offline, easily auditable implementation over a large dataset or an external API, so that every stage of the ML pipeline is visible and testable within the codebase itself.

2. Problem Statement

Manually browsing a large movie catalogue to find something similar to a film already enjoyed is time-consuming and depends on a platform's opaque, black-box recommendation logic. Many student projects address recommendation with collaborative filtering, which requires large volumes of real user-rating data that a solo academic project cannot realistically collect. CineMatch instead solves this with a lightweight, transparent, locally-runnable recommender that works purely from movie content -- genre tags and plot text -- rather than from user behaviour data.

3. Functional Requirements

- **Module 1 -- Data Ingestion & Preprocessing:** load the movie catalogue from CSV, validate its structure, combine genre and overview text per movie, and vectorize it with TF-IDF.
- **Module 2 -- Recommendation Engine (ML core):** compute a cosine-similarity matrix over the TF-IDF vectors and, given a movie title, return the top-N most similar movies with their similarity scores.
- **Module 3 -- CLI & Analytics:** an interactive command-line interface that accepts a movie title, a 'list' command, and a 'stats' command that reports genre-distribution analytics over the loaded catalogue.

Input/Output structure: input is a free-text movie title typed at the prompt; output is a ranked table of recommended titles, genres, and similarity scores (0-1). **Workflow:** Load dataset → preprocess & vectorize → compute similarity matrix once → repeatedly accept user commands until the user exits.

4. Non-Functional Requirements

Requirement	How CineMatch addresses it
Performance	Cosine-similarity matrix computed once at startup and cached; each recommendation is an O(1) row lookup thereafter.
Usability	Guided CLI prompts and fuzzy title matching tolerate typos and case differences.
Reliability	Custom exceptions (DataLoadError, MovieNotFoundError) replace raw stack traces with actionable, user-facing messages.
Maintainability	Each pipeline stage lives in its own single-purpose module (ingest / preprocess / recommend / report / interact).
Scalability	Swapping data/movies.csv for a larger catalogue needs no code changes, only a config.py path update.

Logging / Monitoring

All key events (dataset load, vectorization, recommendation requests, errors) are logged to cinematch.log with timestamps.

5. System Architecture

The system is organised into clearly separated layers: data access (data_loader.py), preprocessing/ML feature engineering (preprocessor.py), the ML core (recommender.py), presentation/reporting (cli.py, analytics.py), and cross-cutting concerns (config.py, logger_config.py). Each arrow below represents a function call and the data passed along with it.

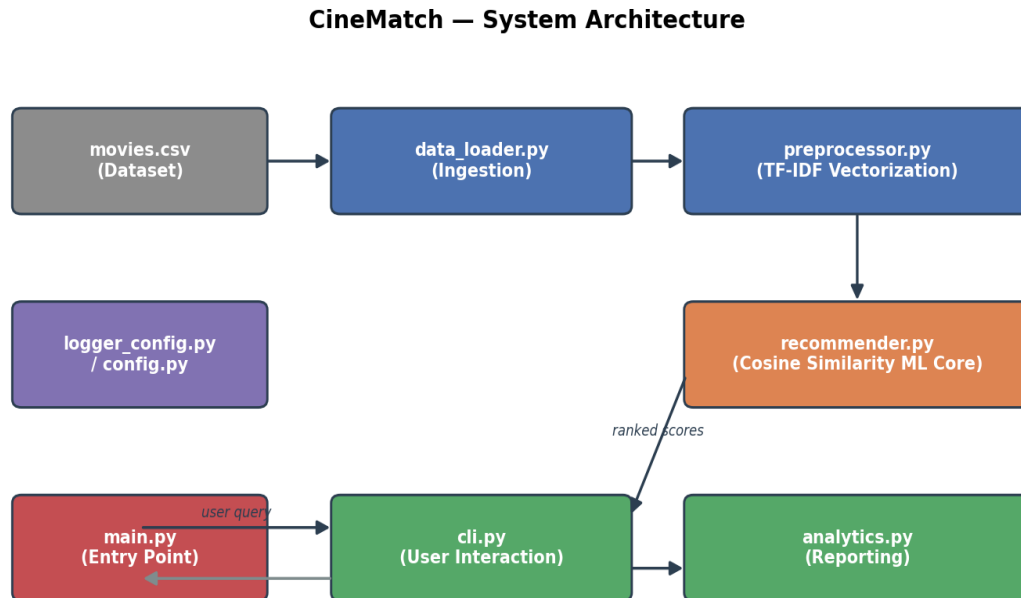


Figure 1: System Architecture Diagram

6. Design Diagrams

6.1 Use Case Diagram

CineMatch — Use Case Diagram

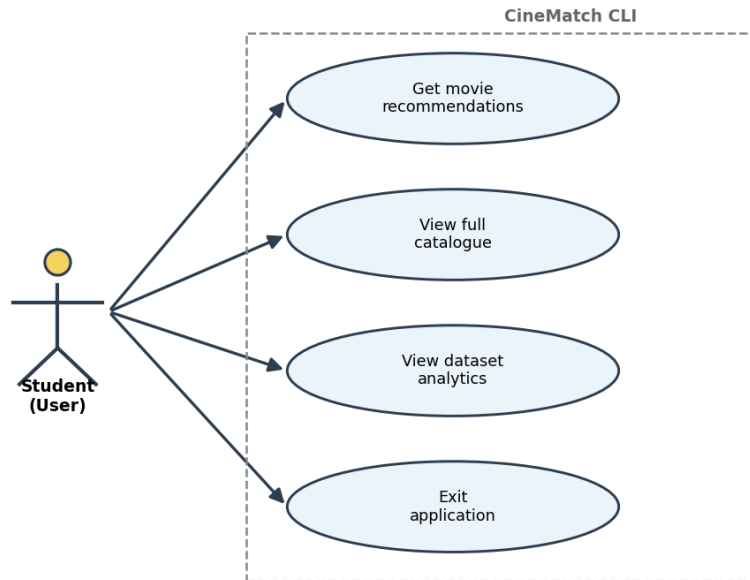


Figure 2: Use Case Diagram

6.2 Process Flow / Workflow Diagram

CineMatch — Workflow

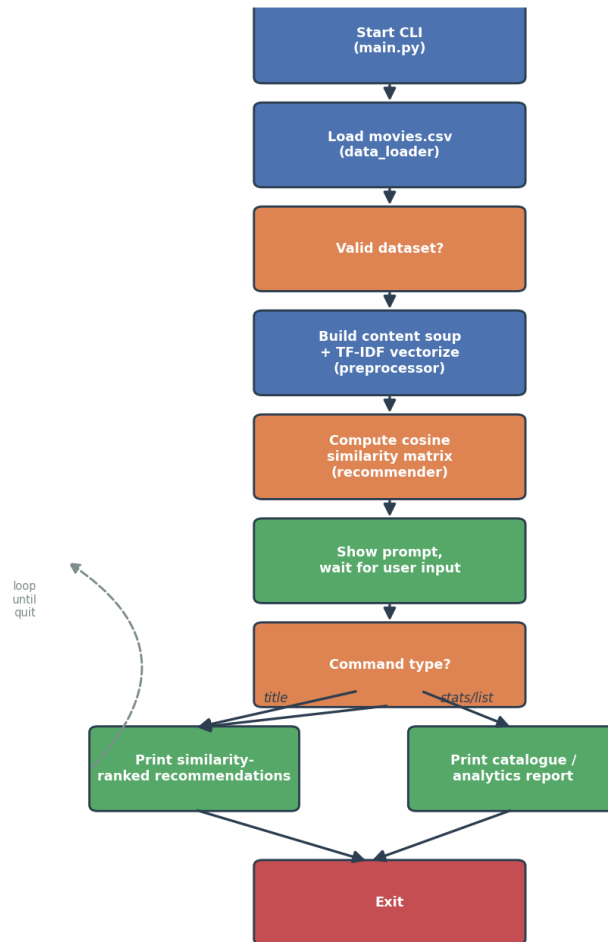


Figure 3: Workflow Diagram

6.3 Class / Component Diagram

CineMatch — Class / Component Diagram (simplified UML)

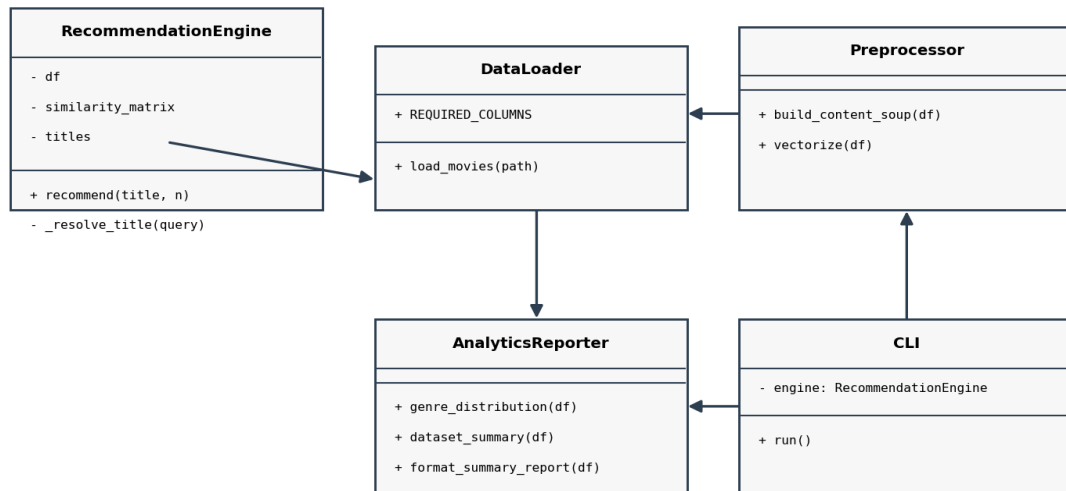


Figure 4: Simplified UML Class / Component Diagram

6.4 Sequence Diagram

CineMatch — Sequence Diagram: 'Get Recommendations'

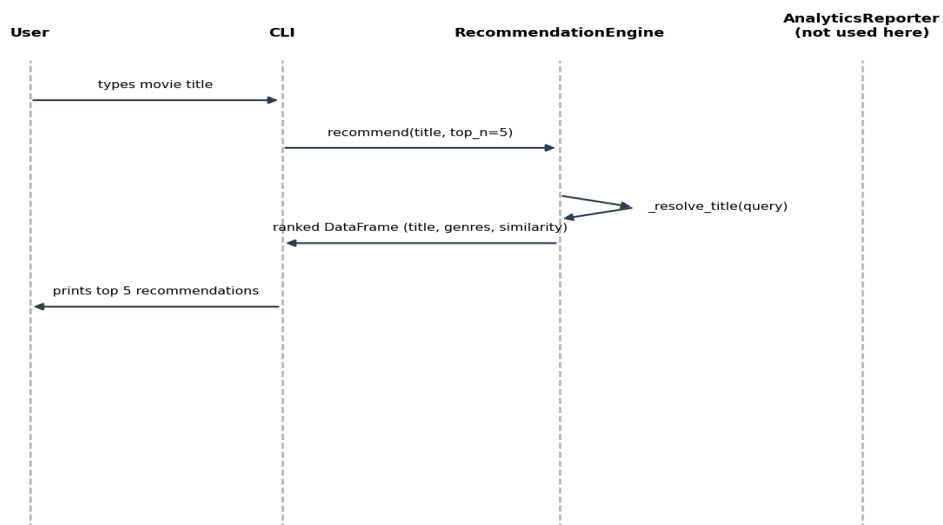


Figure 5: Sequence Diagram -- 'Get Recommendations' happy path

Note on storage/ER diagrams: CineMatch reads a static bundled CSV file and holds all data in memory (pandas DataFrame) for the duration of a session; there is no database or persistent storage layer, so an ER diagram / schema design is not applicable to this project.

7. Design Decisions & Rationale

- **Content-based filtering over collaborative filtering:** collaborative filtering needs a large user-item rating matrix that does not exist for a solo project; content-based filtering only needs item metadata, which can be curated directly.
- **TF-IDF over simple keyword matching:** TF-IDF down-weights common words (e.g. 'the', 'man') and up-weights terms that are distinctive to a particular movie's description, giving more meaningful similarity scores than raw keyword overlap.
- **Genre text repeated 3x in the 'content soup':** genre tags are a stronger, cleaner similarity signal than free-text overviews, so they are deliberately weighted more heavily before vectorization (see `preprocessor.py`).
- **Cosine similarity over Euclidean distance:** cosine similarity is invariant to document length, which matters here because overview lengths vary considerably across movies while genre-tag counts do not.
- **Similarity matrix pre-computed once at startup:** trades a small amount of startup time for $O(1)$ lookups during the interactive session, which is the right trade-off for a tool used interactively and repeatedly.
- **Dataset description:** a hand-curated 40-movie catalogue spanning 16 genre tags was used instead of a very large public dataset, so the whole pipeline runs instantly with no download step and every recommendation can be manually sanity-checked against the source data.
- **Model selection rationale:** TF-IDF + cosine similarity was chosen over a pre-trained embedding model (e.g. sentence-transformers) because it needs no external model download, runs deterministically offline, and is simple enough to explain and defend line-by-line -- appropriate for a course project whose goal is demonstrating understanding of the underlying ML technique.
- **Evaluation methodology:** correctness was checked two ways -- (a) automated unit tests asserting that recommended movies share genre tags with the query movie and that a movie never recommends itself, and (b) manual inspection confirming that, e.g., querying a science-fiction film returns other science-fiction films as its top matches (see Section 10, Screenshots).

8. Implementation Details

The project is implemented in Python 3 using pandas for data handling and scikit-learn for the ML pipeline (TfidfVectorizer, cosine_similarity). The codebase is split into 9 files across a `src/` package, each with a single responsibility (see Section 6.3). Configuration (dataset path, default recommendation count) is centralised in `config.py` so behaviour can be tuned without touching logic files. All significant runtime events are written to a rotating text log (`cinematch.log`) via a shared logger factory in `logger_config.py`, satisfying the Logging / Monitoring requirement. User-facing errors are raised as custom exception types (`DataLoadError`, `MovieNotFoundError`) and caught at the CLI boundary so the terminal never shows a raw Python traceback to the end user.

Key modules and file count (satisfies the 5-10 meaningful modules/files requirement):

- `main.py` -- entry point
- `src/config.py` -- configuration
- `src/logger_config.py` -- shared logger

- `src/data_loader.py` -- dataset ingestion & validation
- `src/preprocessor.py` -- TF-IDF feature engineering
- `src/recommender.py` -- ML recommendation engine
- `src/analytics.py` -- dataset analytics/reporting
- `src/cli.py` -- interactive command-line interface
- `tests/test_recommender.py` -- automated test suite

9. Screenshots / Results

Below is a captured terminal session showing a recommendation query for 'Interstellar' (a science-fiction/drama film) followed by the 'stats' analytics command. Note that all five returned recommendations are science-fiction and/or adventure titles, confirming the recommender is picking up on genuine thematic similarity rather than noise.

```

terminal — python main.py

|               CineMatch v1.0               |
| Content-Based Movie Recommender CLI |
|_____|

Commands:

<movie title>  -> get recommendations similar to that movie
list           -> show all movies in the catalogue
stats          -> show dataset analytics
quit / exit    -> leave CineMatch

Loaded 40 movies. Type a movie title to begin.

cinematch>
Because you liked 'Interstellar', you might enjoy:

1. The Martian [Sci-Fi Adventure Drama] (similarity: 0.479)
2. Avengers Endgame [Action Adventure Sci-Fi] (similarity: 0.374)
3. Iron Man [Action Adventure Sci-Fi] (similarity: 0.357)
4. Gravity [Sci-Fi Drama Thriller] (similarity: 0.354)
5. Guardians of the Galaxy [Action Adventure Comedy Sci-Fi] (similarity: 0.349)

cinematch> =====

CineMatch Dataset Analytics

=====

Total movies in catalogue : 40
Distinct genre tags      : 16
Most common genre        : Drama (26 movies)

-----

Top genres:

Drama      26
Sci-Fi     14
Adventure   10
Comedy      9
Thriller    8
Action      6
Biography   6
Mystery     5

=====

cinematch> Goodbye!

```

Figure 6: Live terminal session

10. Testing Approach

An automated pytest suite (tests/test_recommender.py) covers the full pipeline with 9 tests: dataset structure/validity, correct error raising on a missing dataset file, correctness of the text-preprocessing step, correct recommendation count and self-exclusion, thematic sensibility of recommendations (genre overlap), fuzzy-matching tolerance for typos, correct exception raising for an unknown title, and correctness of the analytics summary. All 9 tests pass:

```
===== test session starts
=====
collected 9 items
```

```
tests/test_recommender.py::test_dataset_loads_with_required_columns PASSED
tests/test_recommender.py::test_missing_dataset_raises_error PASSED
tests/test_recommender.py::test_content_soup_is_lowercase PASSED
tests/test_recommender.py::test_recommend_returns_requested_count PASSED
tests/test_recommender.py::test_recommend_is_thematically_sensible PASSED
tests/test_recommender.py::test_fuzzy_matching_tolerates_typos PASSED
tests/test_recommender.py::test_unknown_movie_raises_custom_error PASSED
tests/test_recommender.py::test_genre_distribution_counts_correctly PASSED
tests/test_recommender.py::test_dataset_summary_shape PASSED
```

```
===== 9 passed in 1.20s
```

```
=====
```

Run with: `python -m pytest tests/ -v`

11. Challenges Faced

- Balancing how much weight to give genre tags versus free-text overviews in the TF-IDF input; giving them equal weight produced recommendations that sometimes matched only on incidental overview wording rather than genre, so genre text was deliberately repeated to increase its influence.
- Handling user typos gracefully without silently guessing wrong -- resolved by using difflib's fuzzy string matching with a similarity cutoff, combined with a clear error message (including example titles) when no reasonable match exists.
- Keeping the ML core (recommender.py) unit-testable independently of the terminal -- resolved by strictly separating I/O (cli.py) from computation (recommender.py, analytics.py), so tests can call the ML logic directly without simulating keyboard input.

12. Learnings & Key Takeaways

Building CineMatch reinforced that a correct, well-scoped classical ML technique (TF-IDF + cosine similarity) can produce genuinely useful results without needing large datasets, GPUs, or external APIs. It also reinforced the value of separating concerns early: because ingestion, feature engineering, the ML core, and presentation were split into distinct modules from the start, each could be tested and debugged independently, which made the fuzzy-matching and error-handling additions straightforward to bolt on later without touching the ML logic itself.

13. Future Enhancements

- Incorporate collaborative filtering once real user-rating data is available, and blend it with the existing content-based scores (a hybrid recommender).
- Replace TF-IDF with sentence embeddings (e.g. from a small transformer model) for deeper semantic matching beyond exact word overlap.
- Wrap the existing recommendation engine in a lightweight web UI while reusing `src/recommender.py` and `src/analytics.py` unchanged.

14. References

- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- scikit-learn documentation -- TfidfVectorizer and cosine_similarity (scikit-learn.org).
- pandas documentation (pandas.pydata.org).
- Python difflib documentation -- SequenceMatcher-based fuzzy string matching (docs.python.org).